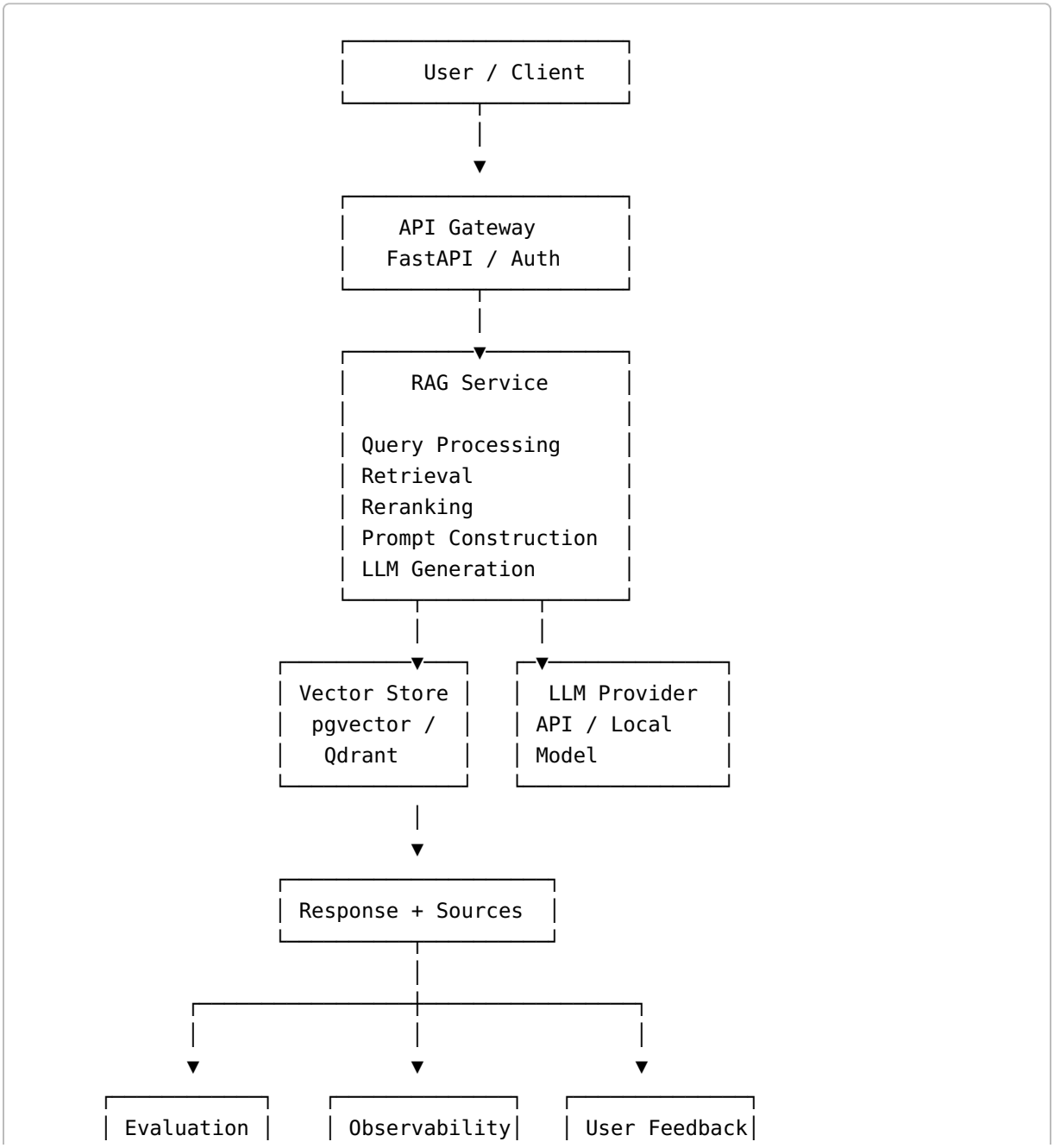
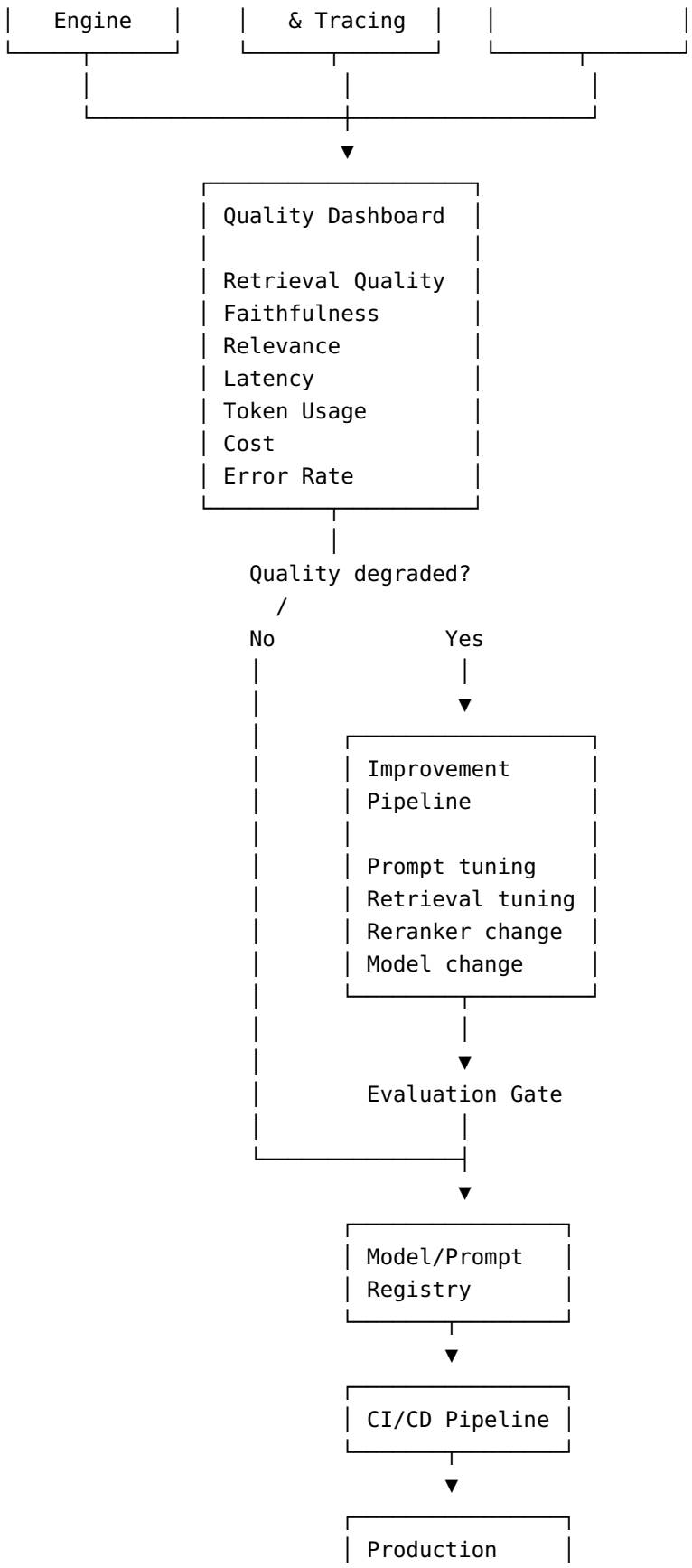
Production RAG System with Evaluation & Observability

A production-grade RAG/LLMOps platform designed to demonstrate modern AI engineering practices beyond a basic RAG chatbot.

1. Concrete System Architecture




Engine
& Tracing
Quality Dashboard
Retrieval Quality
Faithfulness
Relevance
Latency
Token Usage
Cost
Error Rate
Quality degraded?
No
Yes
Improvement Pipeline
Prompt tuning
Retrieval tuning
Reranker change
Model change
Evaluation Gate
Model/Prompt Registry
CI/CD Pipeline
Production

```
                                  | Deployment     |
                                  └────────────────┘
```

2. Document Ingestion Pipeline

Documents should not be manually uploaded directly into the vector database.

Build a separate ingestion pipeline:

```
Documents
   |
   ▼
Object Storage
(S3 / MinIO)
   |
   ▼
Document Parser
   |
   ▼
Cleaning
   |
   ▼
Chunking
   |
   ▼
Embedding Model
   |
   ▼
Vector Database
   |
   ▼
Metadata / Version Store
```

Each document should maintain metadata such as:

```
{
  "document_id": "doc_102",
  "version": "v3",
  "source": "engineering_handbook.pdf",
  "chunk_id": "doc_102_chunk_18",
  "created_at": "2026-09-20",
  "embedding_model": "embedding-model-v2"
}
```

This allows you to reproduce and audit exactly which document version produced a response.

3. Online RAG Pipeline

For every user query:

```
User Query
    |
    ▼
Query Validation
    |
    ▼
Query Transformation
    |
    ▼
Embedding
    |
    ▼
Vector Search
    |
    ▼
Top-K Documents
    |
    ▼
Reranker
    |
    ▼
Top-N Context
    |
    ▼
Prompt Builder
    |
    ▼
LLM
    |
    ▼
Response + Citations
```

The response should ideally contain citations to the retrieved source chunks.

For example:

```
{
  "answer": "The service supports OAuth 2.0...",
  "sources": [
```

```
    {
      "document": "authentication-guide.pdf",
      "page": 14,
      "chunk_id": "auth_14_03"
    }
  ]
}
```

4. Evaluation Pipeline

This is one of the most important parts of the project.

Create a fixed evaluation dataset:

```
eval_dataset/
├── questions.json
├── expected_context.json
└── expected_answers.json
```

Run the complete RAG pipeline against the dataset.

Evaluate:

Retrieval

- Recall@K
- Precision@K
- MRR
- Context relevance

Generation

- Answer relevance
- Faithfulness / groundedness
- Citation correctness
- Context utilization

System

- P50/P95 latency
- Error rate
- Token consumption
- Cost per request

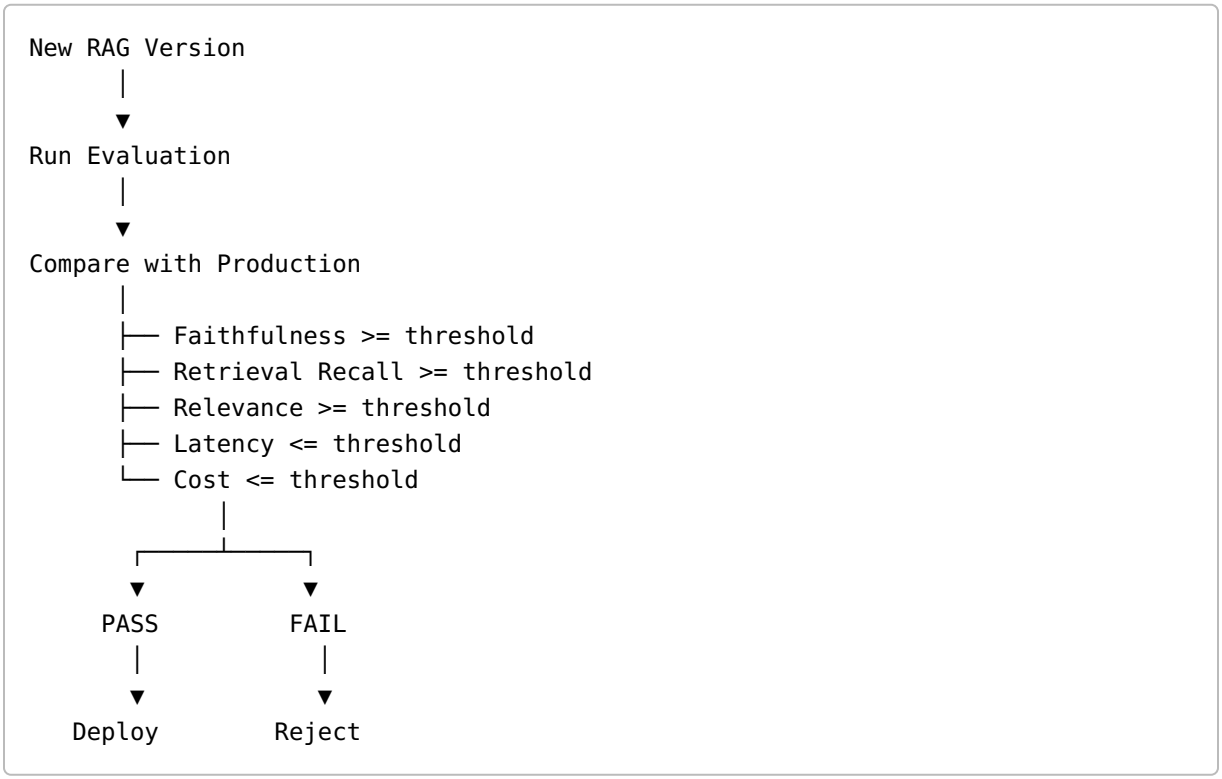
Then produce something like:

```
RAG Evaluation
──────────────────────────────
Recall@5             0.91
MRR                  0.84
Context Relevance    0.89
Faithfulness         0.94
Answer Relevance     0.92
Citation Accuracy    0.96
P95 Latency          1.84 sec
Cost / 1K Requests   $X.XX
```

5. Quality Gate

Do not deploy a new version simply because it was trained or changed.

Create explicit deployment criteria:



This is what turns evaluation into an actual **MLOps/LLMOps workflow** rather than a notebook metric.

6. Observability Architecture

Use distributed tracing around every request:

```
Request
  │
  ├── Query processing       20ms
  │
  ├── Embedding              35ms
  │
  ├── Vector search          80ms
  │
  ├── Reranking              60ms
  │
  ├── LLM generation         900ms
  │
  └── Response               15ms
```

Track:

Application metrics

- Requests/sec
- Error rate
- P50/P95/P99 latency
- Timeout rate

Retrieval metrics

- Number of retrieved chunks
- Retrieval latency
- similarity scores
- empty retrieval rate

LLM metrics

- Input tokens
- Output tokens
- latency
- cost
- model/version

Quality metrics

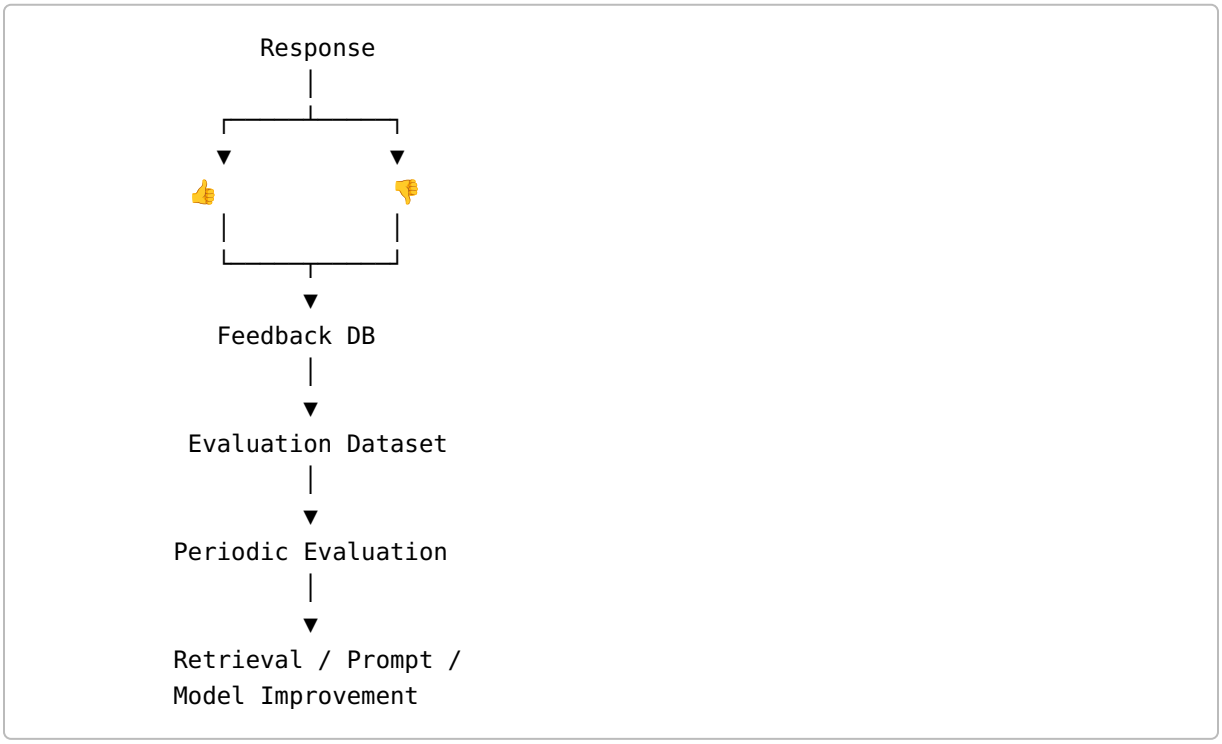
- Faithfulness
- Relevance
- Citation accuracy
- User feedback

Use tools such as:

```
OpenTelemetry
      ↓
Tracing Backend
      ↓
Grafana
      ↓
Production Dashboard
```

7. Feedback Loop

Collect feedback from users:



This creates a real production feedback loop instead of treating the RAG system as static.

8. CI/CD Architecture

```
Developer
    |
    ▼
Git Push
    |
    ▼
GitHub Actions
    |
```

```
    ├── Unit Tests
    ├── Integration Tests
    ├── Data/Schema Tests
    ├── Security Checks
    ├── Build Docker Image
    │
    ▼
RAG Evaluation
    │
    ▼
Quality Gate
   /
 FAIL  PASS
  │      │
  │      ▼
  │   Container Registry
  │      │
  │      ▼
  │   Deployment
  │
  └── Stop
```

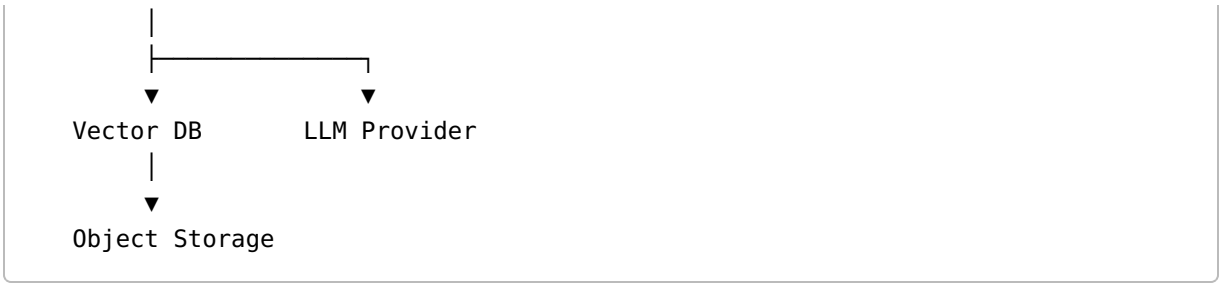
9. Deployment Architecture

Start locally:

```
Docker Compose
├── FastAPI
├── PostgreSQL
├── pgvector
├── Redis
├── Evaluation Service
└── Observability Stack
```

Then move toward:

```
                Cloud
                  │
            Load Balancer
                  │
             Kubernetes
                  │
        ┌─────────┼─────────┐
        ▼         ▼         ▼
    RAG API    Eval API   Ingestion
```



You can eventually add:

- Kubernetes
- Helm
- Terraform
- AWS/GCP
- autoscaling
- blue/green or canary deployments

10. Recommended Technology Stack

Core

- Python
- FastAPI
- PostgreSQL
- pgvector or Qdrant
- Docker

RAG

- Sentence Transformers / embedding model
- Reranker
- LLM API or local open-source model

Evaluation

- Ragas or custom evaluation framework
- LLM-as-a-judge where appropriate
- curated golden dataset

Observability

- OpenTelemetry
- Prometheus
- Grafana
- Langfuse or equivalent LLM observability platform

MLOps / LLMOps

- MLflow where useful for model/experiment tracking
- DVC for dataset/version management if the project benefits from it
- GitHub Actions
- Docker Registry

Deployment

- Kubernetes
- Helm
- Terraform
- AWS/GCP

Don't implement every technology from day one. The architecture is the target; build it progressively.

11. Final Resume Positioning

The project should ultimately be described as:

> **RAGOps — Production RAG Platform with Evaluation & Observability**

Description:

> Engineered an end-to-end production RAG platform featuring versioned document ingestion, hybrid/vector retrieval, reranking, automated retrieval and generation evaluation, LLM tracing, latency/token/cost monitoring, CI/CD quality gates, and feedback-driven continuous improvement.

The important distinction is that the project demonstrates **engineering a reliable AI system**, not simply building a chatbot.

Development progression

```
Stage 1
Basic RAG
   ↓
Stage 2
Evaluation
   ↓
Stage 3
Tracing + Observability
   ↓
Stage 4
CI/CD + Quality Gates
   ↓
```

```
Stage 5
Production Deployment
   ↓
Stage 6
Feedback Loop
   ↓
Stage 7
Continuous Improvement
   ↓
Stage 8
Kubernetes + Cloud + Canary Deployment
```

The final system should be something you can actually **demonstrate live and defend technically in an interview**, rather than a collection of tools added purely for the resume.